

closure_evidence_20260925

BOB-191 — UNANALYSED- extension registry: closure evidence

Revision: 1 **Last modified:** 2026-09-25T00:00:00Z **Status:** fix implemented + tested + live-scanned; item NOT closed in DB (per dispatch instructions) **Scope:** BOB-191 (fail-open scanner MATCHER hole blind to enumerated non-catch languages) **Authority:** §11.4.6, §11.4.102, §11.4.115, §11.4.201(6)/(7)(b), §11.4.224, §11.4.250, §11.4.252

0. Critical correction to the dispatch prompt (§11.4.6, stated up front)

The dispatch prompt named `scripts/pre_build/cm_no_fail_open_skip_analyzer.py / scripts/pre_build/check_cm_no_fail_open_skip.sh / tests/pre_build/test_check_cm_no_fail_open_skip.sh` as “the CM-NO-FAIL-OPEN-SKIP gate” for BOB-191. **This is a different gate for a different anchor.** Reading BOB-191’s own stored DB description (queried first, in full, per the dispatch instructions) shows the gate it actually concerns is `constitution/scripts/gates/cm_dangerous_combination_fail_closed.sh` — the **CM-DANGEROUS-COMBINATION-FAIL-CLOSED** gate for §11.4.252, driven by invariant 39 at `scripts/pre_build_verification.sh:1761` (line numbers as of this session, pre-edit; BOB-191’s own round-2 investigation evidence doc, `docs/qa/BOB-191/scanner_blindness_investigation_20260826.md`, independently confirms this and additionally notes that `check_cm_no_fail_open_skip.sh` did not exist on any branch as of 2026-08-26).

cm_no_fail_open_skip_analyzer.py is real and does exist (it was added after 2026-08-26, for **BOB-161**, an unrelated defect: it AST-scans **test files** for pytest.skip/ab_skip calls that convert an ANSWERED sink-side response into a skip, per §11.4.69 — a Python/shell, test-corpus-only concern). It has no relationship to BOB-191’s production-code, multi-language, §11.4.252 scope and was **not modified** by this session — touching it would have conflated two orthogonal defects under one item.

All work below targets the gate BOB-191’s own DB record actually names.

1. The 5 languages, and what the live gate does with each

BOB-191’s stored round-2 investigation table lists (go/rs/rb/c/h) as the blind bucket (5 tokens) without empirically settling whether the C++ family (cc/cpp/h/hpp) is genuinely blind or merely grouped loosely with c. This session re-ran the control-needle methodology live, against the CURRENT gate, on ALL FIVE candidates, including the one the prior investigation did not settle:

Extension	Needle planted	Live gate verdict (before this fix)	Structurally blind?
.go	empty if err != nil {} + _ = riskyOp()	PASS – no ... found (rc=0)	YES
.rs	let _ = risky_op() + Err(_) => {}	PASS – no ... found (rc=0)	YES
.rb	rescue StandardError + comment-only body	PASS – no ... found (rc=0)	YES
.c	ignored return + empty if (ret != 0) {}	PASS – no ... found (rc=0)	YES
.cpp	genuine empty catch (const	FAIL – swallowed	NO — already caught

Extension	Needle planted	Live gate verdict (before this fix)	Structurally blind?
	<code>std::exception& e) {}</code>	<code>exception ... :5 (rc=1)</code>	
<code>.h</code> (C-style content)	same C-style empty-if shape as <code>.c</code>	PASS — no ... found (rc=0)	content-dependent, same as <code>.c</code>

Finding, corrected from BOB-191’s own imprecise “5” figure (§11.4.6/§11.4.226 — new evidence stated, not silently overriding the prior record): the gate’s shape-A matcher (`grep -nE 'catch[[:space:]]*\([^)]*\)[[:space:]]*\{'`) runs over **every** enumerated extension, unconditioned by extension — it is not gated to a subset. It is genuinely idiom-correct for C++ (`catch (...)` { **is** real C++ syntax) and the live needle proves it fires correctly on C++. It cannot structurally match Go/Rust/Ruby/C, whose native error-handling idioms contain no `catch` keyword at all. `.h` is extension-ambiguous (a C header vs. a C++ header) and its outcome is **content-dependent**, exactly like `.c` — not a fifth structurally-blind extension in its own right.

Precisely four language families are structurally blind by construction: Go, Rust, Ruby, C. C++ (`cc/cpp/h/hpp`) is NOT blind and was correctly left uncovered by the new registry (see §3, negative control L91).

2. Live repo scope check (before deciding the fix shape)

```
$ for ext in go rs rb c cc cpp h hpp; do
  find . -prune-excludes... -name "*.${ext}" | wc -l
done
go: 124 files
rs: 0 files
rb: 0 files
c: 0 files
cc: 0 files
cpp: 0 files
h: 0 files
hpp: 0 files
```

Go is the **only** blind language with real code in this repo (124 files, mostly qBitTorrent-go, the language of qbittorrent-proxy-go and boba-jackett — the service owning encrypted tracker credentials on port 7189, per BOB-191’s own IMPACT statement). Rust/Ruby/C/C++ have **zero** files anywhere in the first-party tree today.

This matches the dispatch’s own stated fallback path: for a blind language with genuinely zero code today, extend the analyzer’s SCOPE so a FUTURE violation is caught, even with nothing to fix today. It also matches BOB-191’s own round-2 remediation direction (§8 point 1 of the investigation doc): “*Analysers registry with an UNANALYSED verdict (the single structural fix) ... Do this before any Go arm.*” — explicitly ordered BEFORE any semantic Go checker, because a naive regex Go arm would immediately reproduce BOB-189 (fail-CLOSED guards flagged as fail-open) in a new language (§11.4.250: the tower is the diagnostic). Writing a real, call-site-aware Go analyser (go/ast or errcheck/staticcheck-based) is out of scope for this item as filed and remains explicitly deferred future work.

3. The fix: an UNANALYSED-extension registry (RED → GREEN)

RED (before fix, this session, against git show HEAD:...)

```
$ bash <pre-fix gate> --root <go-only fixture> --quiet
  CM-DANGEROUS-COMBINATION-FAIL-CLOSED: PASS – no swallowed-
exception, silent-default-return or credential-default-to-literal
anti-patterns found (§11.4.252)
rc=0
```

(identical for .rs/.rb/.c fixtures — no “UNANALYSED” concept existed; the marker string is absent by construction; the bare unqualified clean-PASS sentence IS present.)

GREEN (after fix)

```
$ bash <fixed gate> --root <go-only fixture> --quiet
△ CM-DANGEROUS-COMBINATION-FAIL-CLOSED: NOTE – 1 file(s) of an
UNANALYSED extension (go rs rb c) were enumerated but have NO
idiom-matching fail-open analyser (...) -- sample: .../needle.go.
This is reported as UNKNOWN, never as a clean scan, for this
extension's fail-open posture (§11.4.6/§11.4.201(6)/
§11.4.250). ...
  CM-DANGEROUS-COMBINATION-FAIL-CLOSED: PASS (PARTIAL COVERAGE) –
no swallowed-exception, silent-default-return or credential-
```

default-to-literal anti-patterns found among the ANALYSED files; 1
UNANALYSED file(s) reported above, NOT included in this clean
claim (§11.4.252)
rc=0

What changed (constitution/scripts/gates/ cm_dangerous_combination_fail_closed.sh)

1. New consumer-overridable env var
DANGEROUS_COMBO_UNANALYSED_EXT (default go rs rb c), matching the
existing DANGEROUS_COMBO_EXT / DANGEROUS_COMBO_EXCLUDE override
pattern (§11.4.28/§11.4.35).
2. `is_unanalysed_ext()` classifies each enumerated file; the main scan
loop tallies `unanalysed_hits` + a bounded (first 10) sample list.
3. The final verdict block now has three branches, not two:
 - real hits present → unchanged FAIL (rc=1).
 - zero hits, but UNANALYSED files present → new `△ ... NOTE` line
+ a PASS (PARTIAL COVERAGE) line that explicitly excludes the
unanalysed files from the clean claim (rc=0, still advisory/non-
blocking — matches this same gate's own existing
`topology_unsupported SKIP-on-exit-0` precedent).
 - zero hits, zero UNANALYSED files → unchanged, original,
unqualified PASS line (byte-identical wording to before this fix).
4. `HEADER_LINES` (used by `--help`) bumped from 565 → 579 to match the
header text added; re-verified against the mutation suite's own P13
self-probe (`HEADER_LINES` names a comment line whose next line is
empty).
5. Header docs (Environment overrides, Exit codes) updated in the
same change (§11.4.18).

What changed (constitution/scripts/gates/ cm_dangerous_combination_fail_closed_mutation_test.sh)

New helper `expect_output_not_contains` (the negative twin of the
existing `expect_output_contains`, same capture-first discipline per
§11.4.201(12)).

New section “96. BOB-191: the UNANALYSED-extension registry”,
fixtures L87-L93 (17 new assertions):

- **L87 (Go), L88 (Rust), L89 (Ruby), L90 (C)** — each: (a) gate still exits
0 (advisory, never blocks), (b) output CONTAINS “UNANALYSED
extension”, (c) output does NOT contain the bare unqualified clean-
PASS sentence.

- **L91 NEGATIVE CONTROL** — C++ (.cpp) with a genuine empty catch block is STILL caught (rc=1) and is NEVER reported UNANALYSED — proves the registry was not over-broadened to swallow a working analyser.
- **L92 NEGATIVE CONTROL** — a plain, fully-analysed clean Python root NEVER prints the UNANALYSED marker, and DOES print the plain unqualified clean-PASS sentence — proves the new branch is reached only when genuinely warranted.
- **L93** — a REAL Python hit alongside an unanalysed Go file in the SAME root still FAILs (rc=1) — proves the new branch can never mask a genuine finding.

Full suite result

```
$ bash constitution/scripts/gates/
cm_dangerous_combination_fail_closed_mutation_test.sh
...
  META PASS — CM-DANGEROUS-COMBINATION-FAIL-CLOSED FAILs-on-
mutation AND PASSes-on-clean for every fixture (§1.1 proof holds)
META_EXIT=0
```

206/206 assertions green (189 pre-existing + 17 new), 0 failures. The pre-existing 189 assertions were re-run unmodified before AND after the fix with identical results — no regression to any existing detection shape (swallowed exception, silent-default-return, credential-default, BOB-189 call-site discrimination, contextlib.suppress family, etc).

4. Caller-level fix (scripts/ pre_build_verification.sh, invariant 39)

The gate fix alone was not sufficient to close the loop end-to-end: invariant 39’s own caller only inspected the gate’s log when its exit code was non-zero, and the new UNANALYSED branch deliberately still exits 0 (it is an honest gap, not a hit — matching this gate’s own precedent for topology_unsupported). Left unchanged, invariant 39 would have kept printing “*no fail-open anti-pattern across N first-party source root(s)*” for a root that is 100% UNANALYSED — reproducing BOB-191’s own “matcher hole prints green” defect one layer up, in the actual pre-build report an operator reads.

Fixed: invariant 39 now reads every root’s log unconditionally for the gate’s NOTE – N file(s) of an UNANALYSED extension line (matched on structure per §11.4.201(9) field-identity, not on the bare △ glyph, which also prefixes unrelated degraded-mode/WAIVED notes), tallies

DANGER_UNANALYSED, and the final message now has three branches (hits+unanalysed / zero-hits-with-unanalysed / genuinely-clean), verified live via an isolated harness (below) before touching the real 57-invariant script.

5. Live full-repo scan (post-fix, real DANGER_ROOTS)

```
DANGER_ROOTS=(. download-proxy/src plugins scripts qBitTorrent-go frontend/src cmd/boba-ctl)
```

Root	Real hits (	FAIL)	UNANALYSED files
. (depth 1)	0		0
download-proxy/src	25		0
plugins	32		0
scripts	2		0
qBitTorrent-go	0		117
frontend/src	0		0
cmd/boba-ctl	0		4

Invariant-39 final summary line (verified via isolated harness against the real DANGER_ROOTS, not the full 57-invariant script — see §4):

```
WARN: CM-DANGEROUS-COMBINATION-FAIL-CLOSED – 59 fail-open hit(s)
across download-proxy/src:25 plugins:32 scripts:2 (ADVISORY, non-
blocking per §11.4.234; un-triaged §11.4.252 backlog, see the
block comment)
      also 121 file(s) UNANALYSED (no idiom-matching analyser
for go/rs/rb/c) across qBitTorrent-go:117 cmd/boba-ctl:4 – UNKNOWN
fail-open posture for those files (§11.4.6/BOB-191)
```

The 59-hit Python backlog (25+32+2) is PRE-EXISTING and UNCHANGED by this session — it is the same un-triaged §11.4.252 backlog invariant 39’s own block comment already names as advisory, first surfaced 2026-08-20, entirely outside BOB-191’s scope. **Not touched, not silently fixed** — per the dispatch instructions, no product code beyond the gate itself was modified.

121 files (117 in qBitTorrent-go + 4 in cmd/boba-ctl) are the newly-honest UNANALYSED count. This session’s fix reports ZERO new FAIL findings in Go — it does not add a Go-specific fail-open detector (see §2: that is explicitly deferred, out of scope, and would itself need call-site

semantic analysis to avoid reproducing BOB-189 in Go). It converts a false “0 hits = clean” into an honest “0 hits, coverage unknown for N files”.

Separately, and NOT acted on in this session: BOB-191’s own round-2 investigation doc already found and filed two REAL Go defects in `qBitTorrent-go/internal/jackettapi/credentials.go` (a credential-DELETE endpoint that discards the `.env-delete` error and still returns 204 No Content, and an UPSERT compensating-rollback whose own rollback error is discarded) as a **separate tracked item, BOB-204** — per §11.4.214 (distinct-but-similar, do not merge) and per this session’s own dispatch instruction (“if so, do NOT silently fix product code beyond the gate itself without flagging it clearly”). Flagged here, not touched.

6. Anti-bluff provenance (§11.4.5/§11.4.69/§11.4.2)

- Every “NOT SEEN” claim above is a live gate invocation, output pasted, not paraphrased.
- The C++ (.cpp) result — the one genuine open question left by BOB-191’s own prior investigation — was independently re-measured, not assumed from the stored “(go/rs/rb/c/h)” shorthand; the correction is stated with its own live evidence, not silently substituted (§11.4.6/§11.4.226).
- RED was captured against the actual pre-fix bytes via `git show HEAD:scripts/gates/cm_dangerous_combination_fail_closed.sh` (read-only; a concurrent process held `.git/modules/constitution/index.lock` at the time, so `git stash` was deliberately avoided rather than force-clearing a lock that might belong to a live operation — §11.4.180 only reaps PROVABLY-dead locks).
- `bash -n` clean on all three touched files; `HEADER_LINES` re-verified against the mutation suite’s own P13 self-probe logic, not merely eyeballed.
- The invariant-39 caller change was verified via an isolated harness extracting the exact block (not a guess at its behaviour) before being treated as correct, in all three branch cases (hits+unanalysed, zero-hits-with-unanalysed, genuinely-clean).
- All scratch fixtures lived under the session scratchpad (`/tmp/claude-1000/.../scratchpad/bob191_needles`) or `/tmp`, never inside the repository; all were removed at the end of the session (§11.4.14).

- No repo file was committed or pushed; the item was not closed in the DB — both per the dispatch instructions.